



FINAL PROJECT

ECOVIBE: APPLICATION FOR BUYING AND SELLING SECOND-HAND CLOTHING

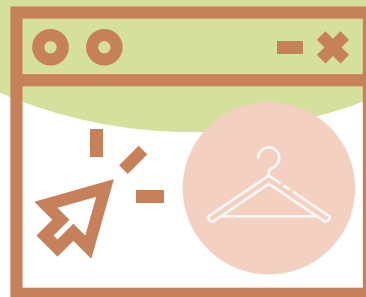
CAROL VELASCO 20231020215
DAVID MUÑOZ 20232020281



ECO VIBE

A Digital Marketplace for Secondhand clothing

A simple and efficient platform for buying and selling clothes.



Uses Java and Python web services for product management.



Designed for a smooth user experience and intuitive navigation.



USER STORIES QUESTIONS

what is your main
interest when
using a virtual
second-
hand clothing
application?

How would you
like the
products to be
displayed in the
app?



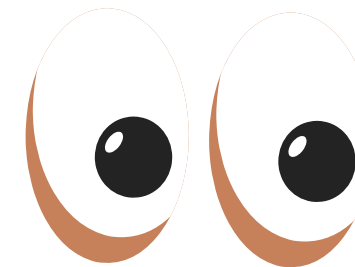
what information
about the
product would
you like to
see provided in
the app?

USER STORIES

"AS A SELLER, I WANT TO BE ABLE TO ADD DETAILED DESCRIPTIONS AND HIGH-QUALITY PHOTOGRAPHS TO SHOW MY PRODUCTS IN THE BEST WAY POSSIBLE"



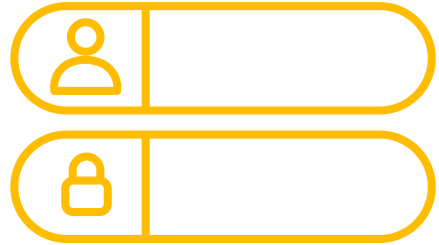
"AS A USER I WANT TO BE ABLE TO SEE THE PRODUCTS WITH PHOTOS AND DETAILED DESCRIPTIONS TO HAVE A CLEAR IDEA OF WHAT I AM BUYING"



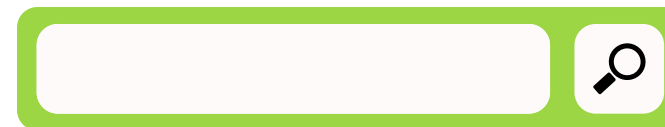
"AS A BUYER, I WANT TO BE ABLE TO SEE DETAILED INFORMATION ABOUT THE PRODUCT, SUCH AS CONDITION AND MEASUREMENTS, TO MAKE AN INFORMED DECISION"



FUNCTIONAL REQUIREMENTS



User Registration:
Username and
password.



Product Search:
Buyers search using
keywords.

Product purchase: Enter
address, bank details and
phone number.



Product Listing: Sellers
upload images,
descriptions and prices.

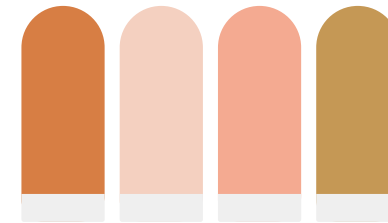


Reviews and Ratings:
Buyers rate products
and sellers.

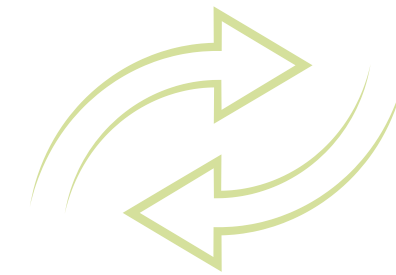


NON-FUNCTIONAL REQUIREMENTS

Usability:
intuitive and
easy to navigate.

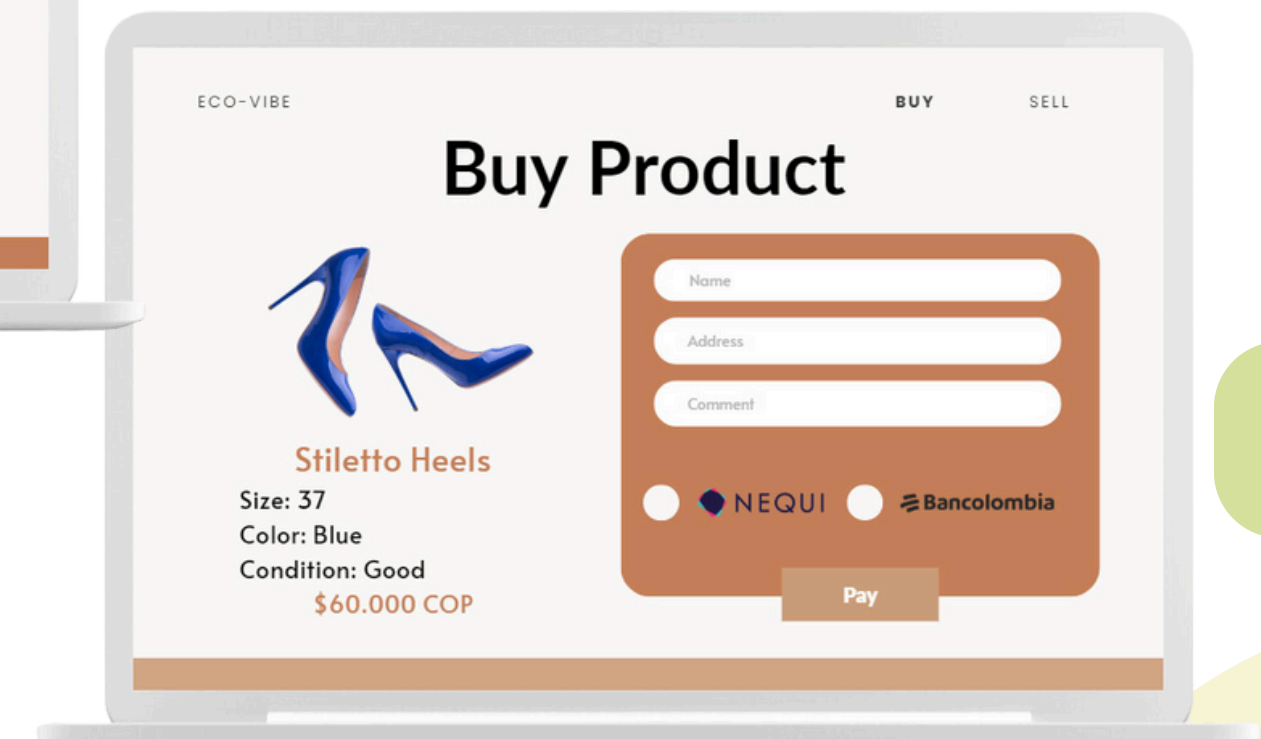
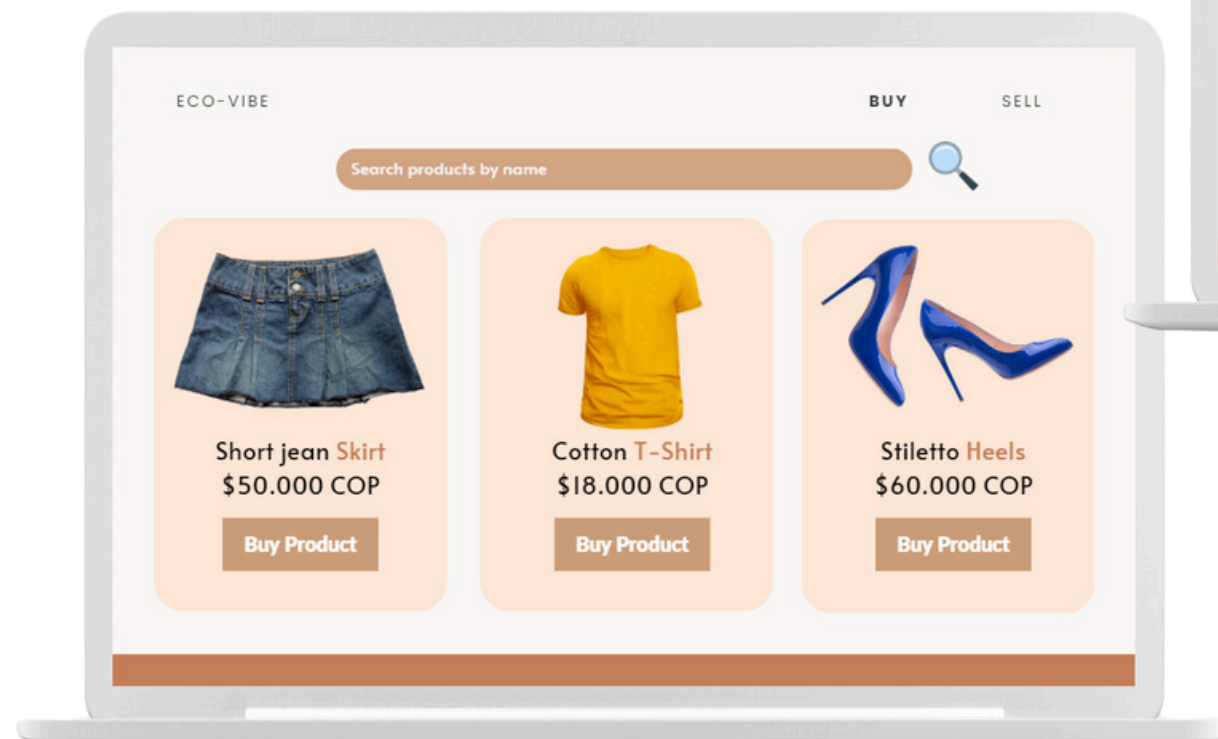
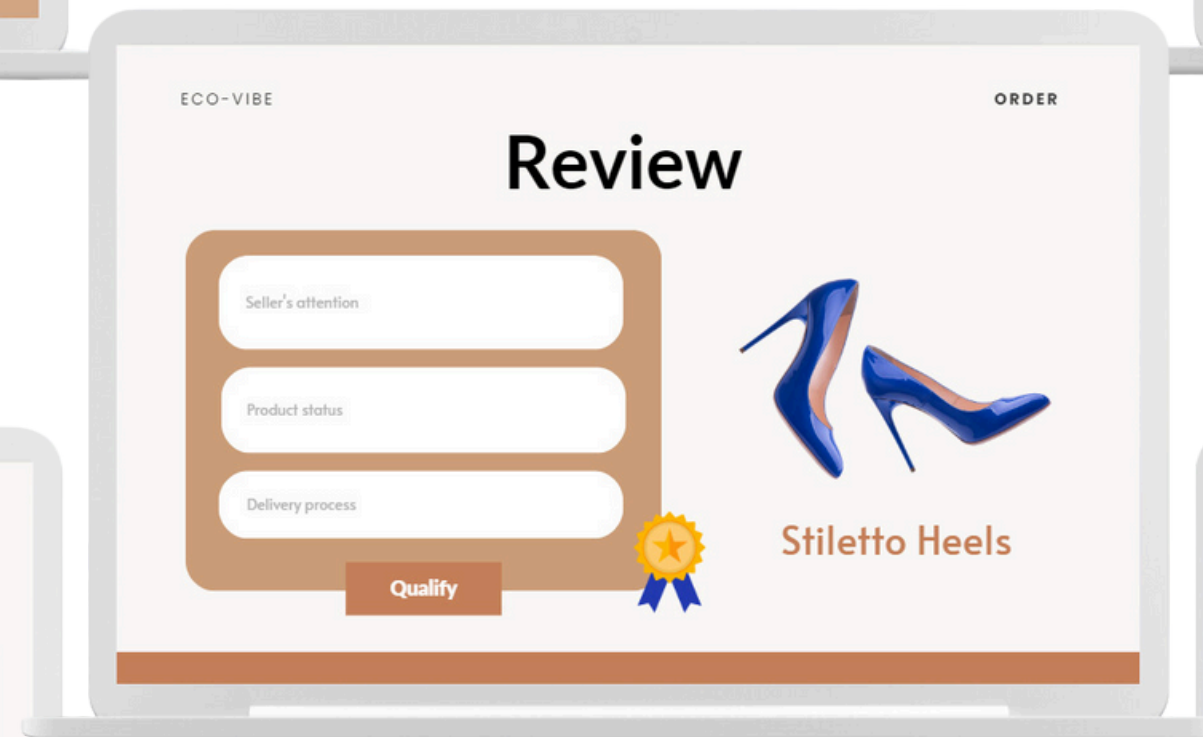
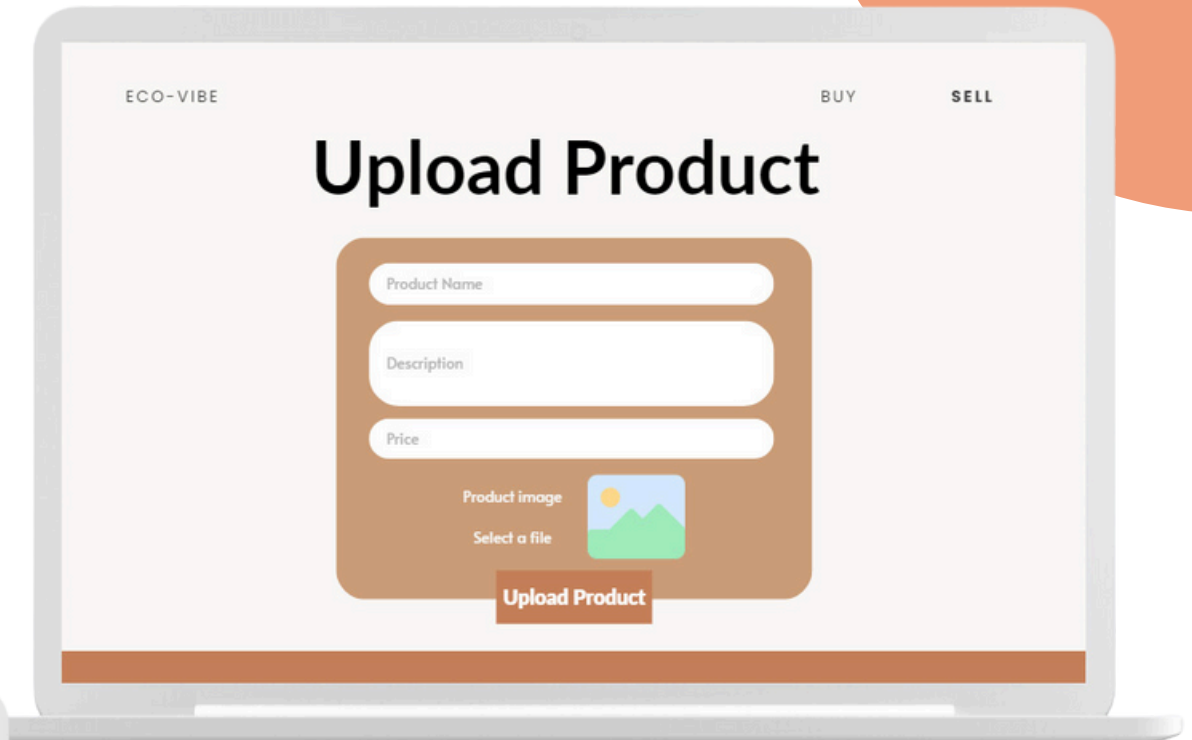
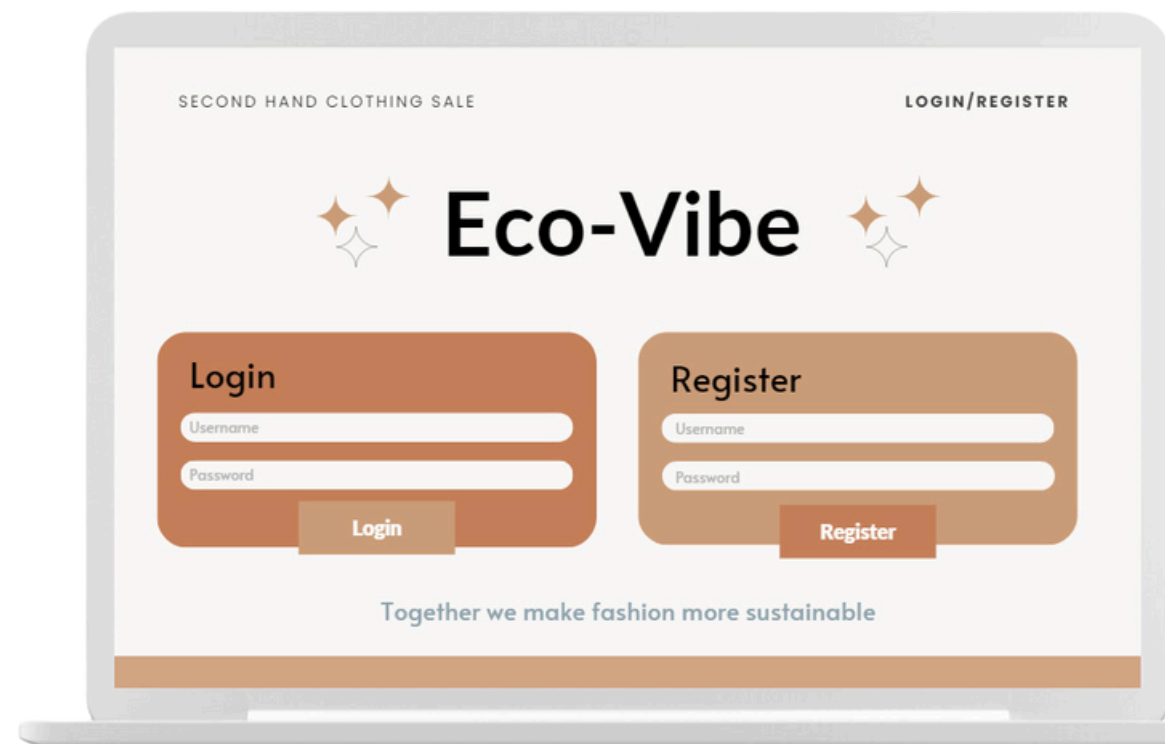


Visual Design:
pleasant colors
and clear layout.

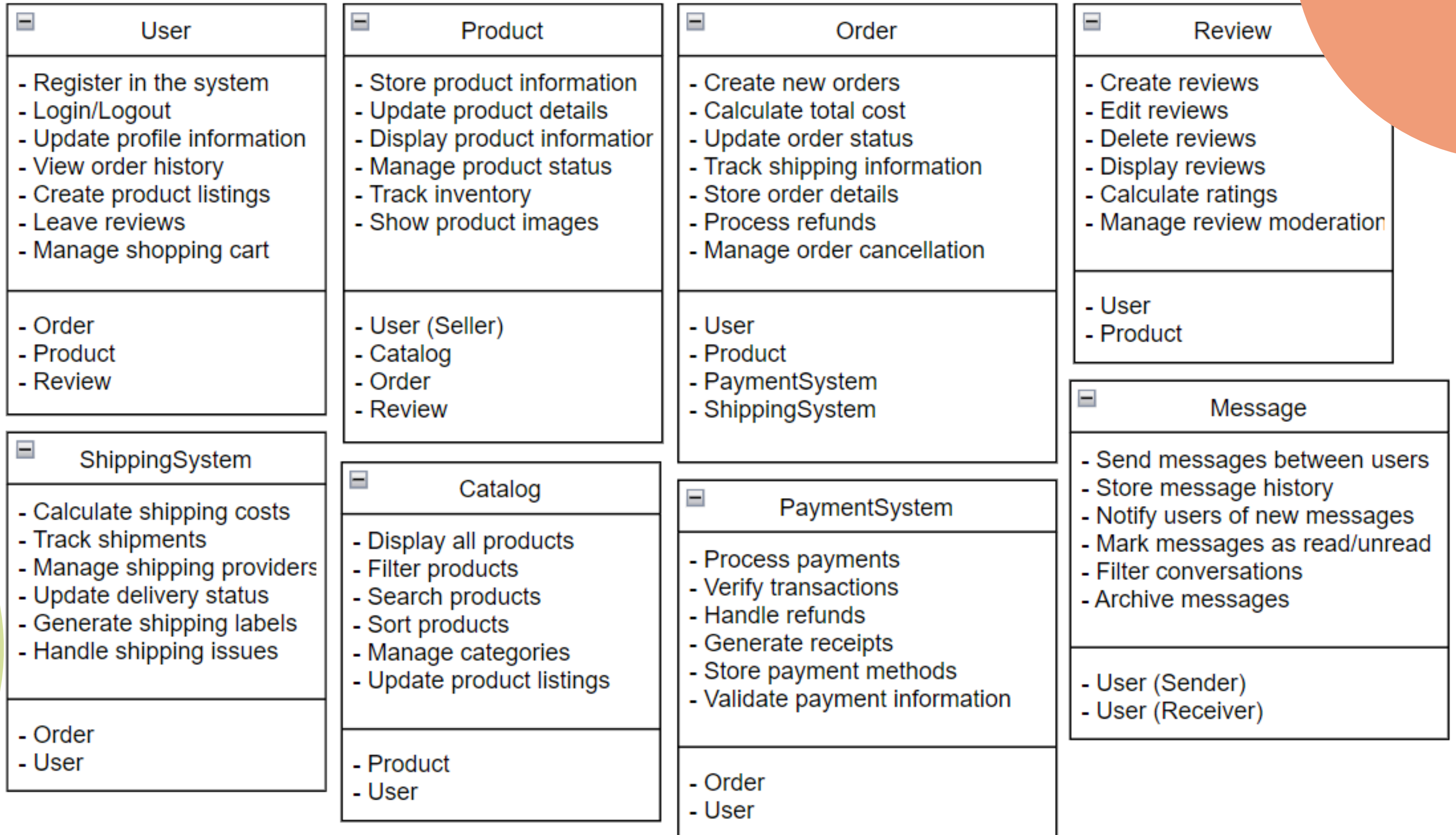


Maintainability:
future updates
without issues.

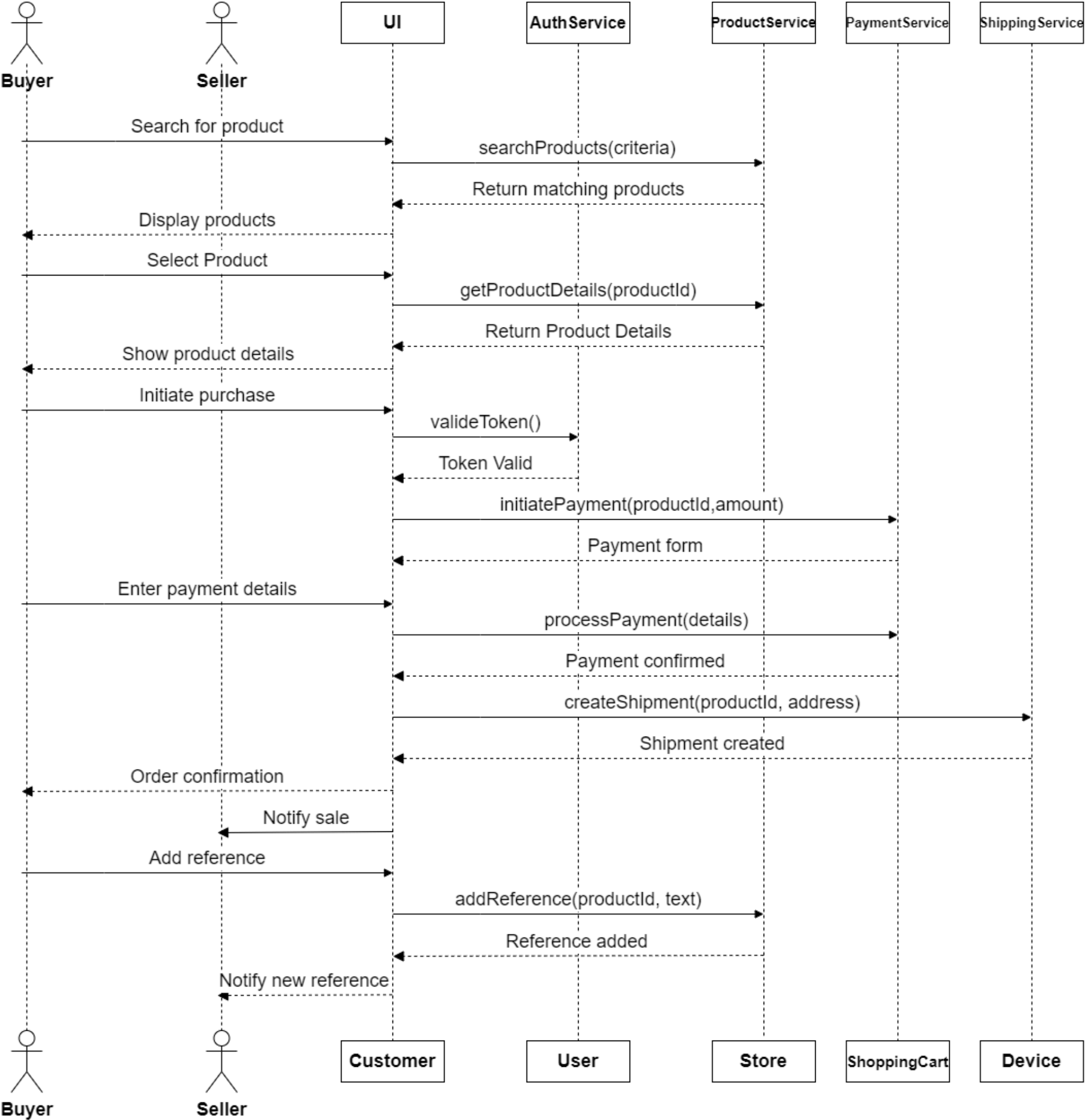
MOCKUPS



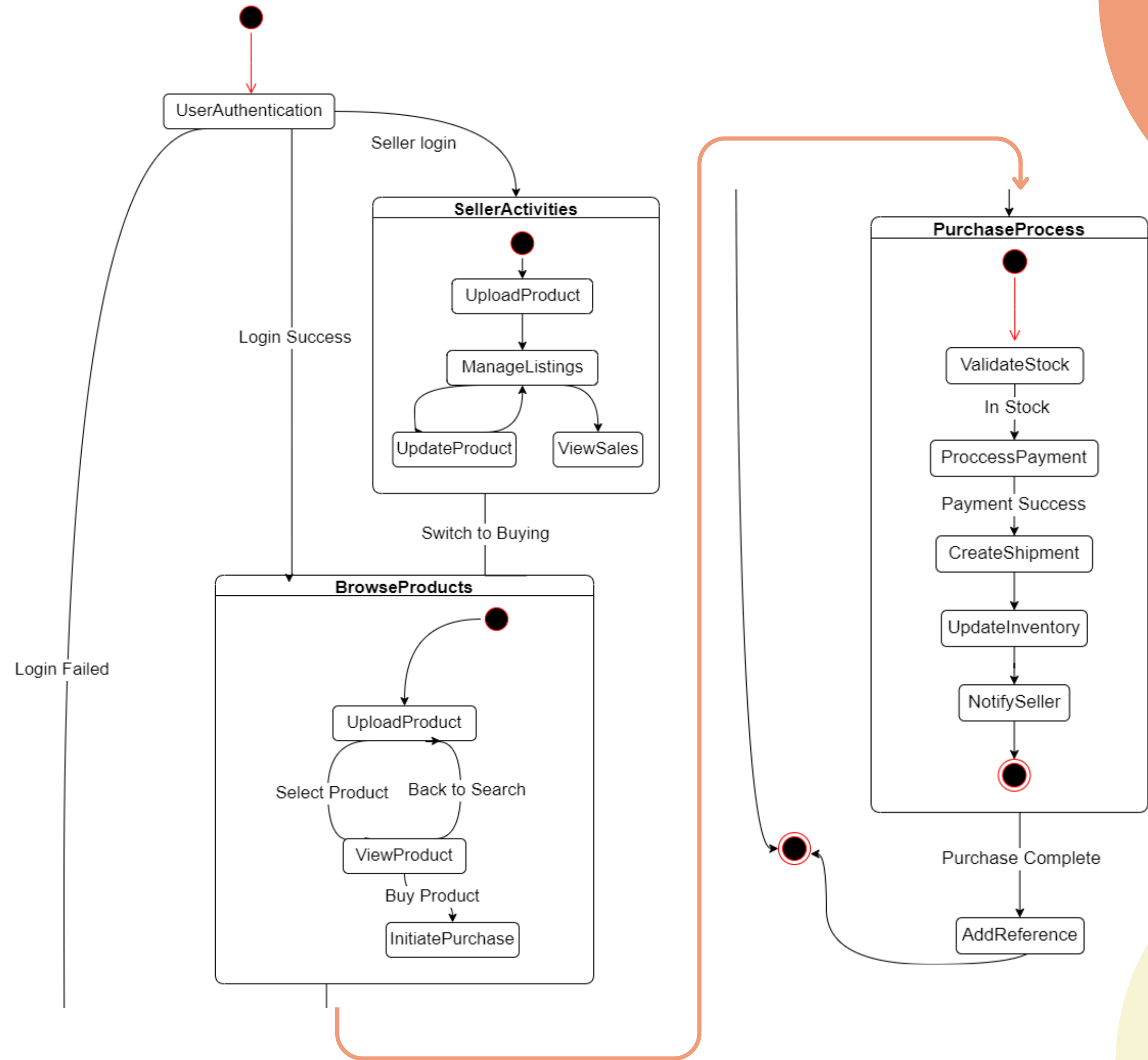
CRC CARDS



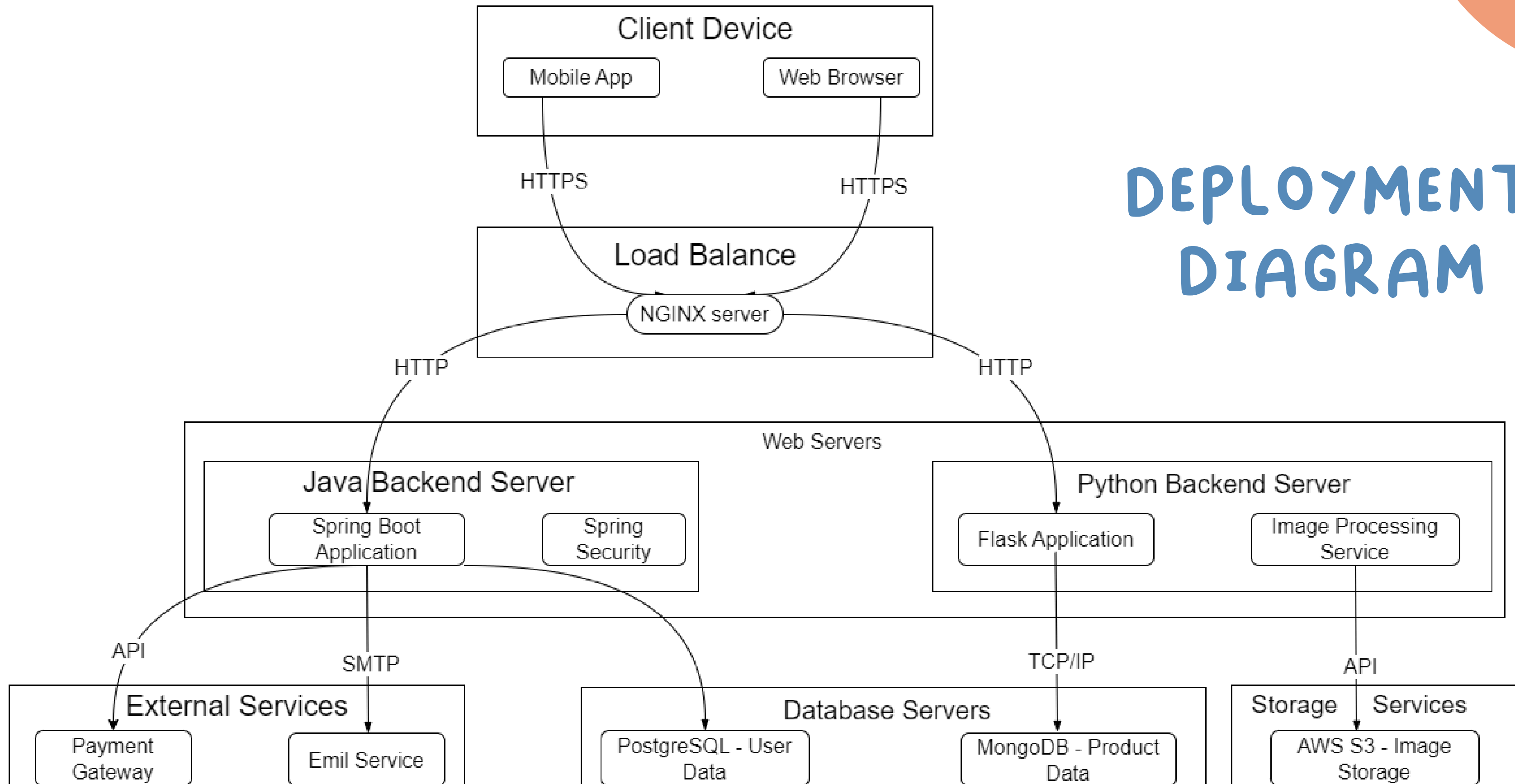
SEQUENCE DIAGRAM



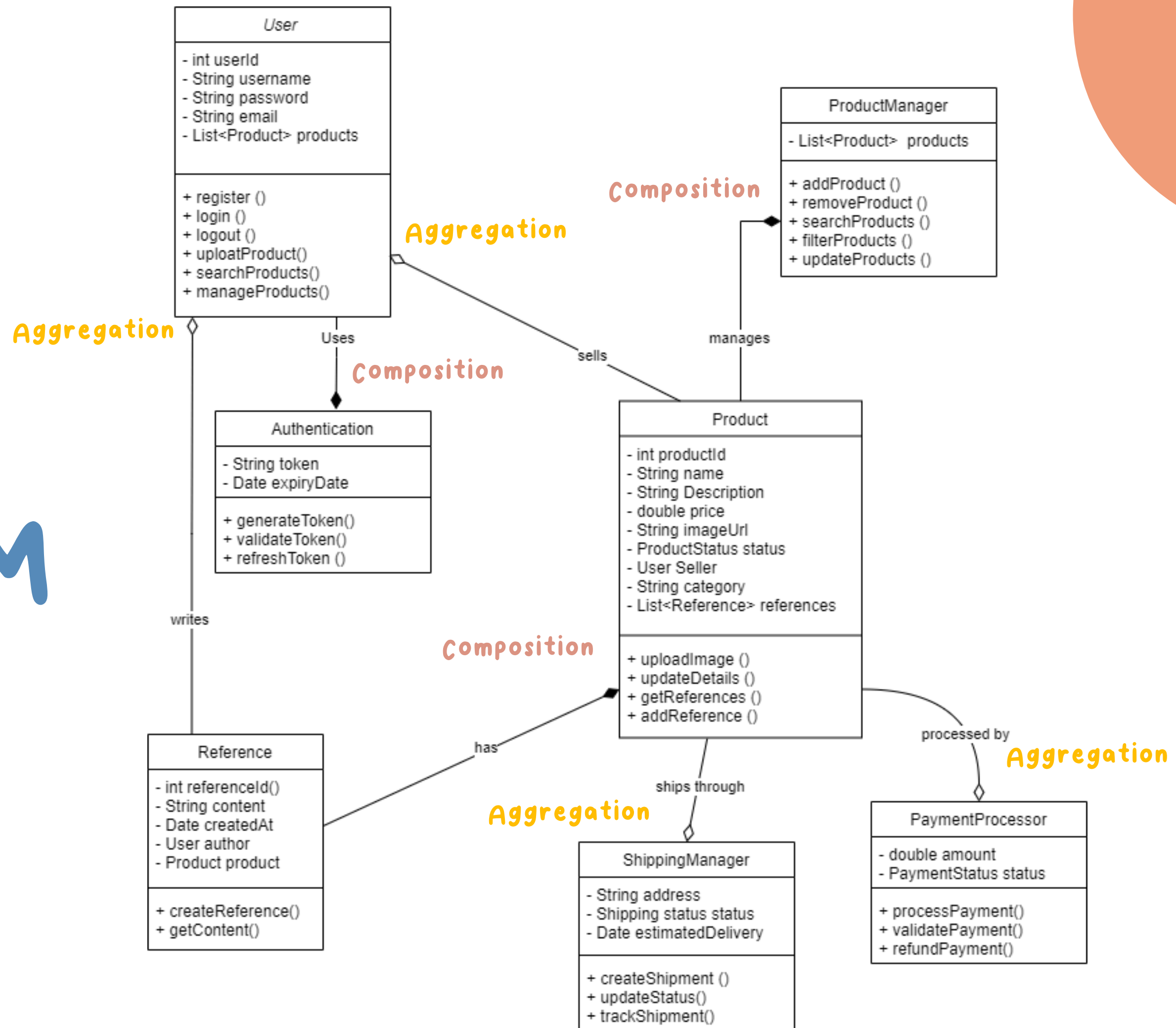
ACTIVITY DIAGRAM



DEPLOYMENT DIAGRAM



CLASS DIAGRAM



BACKEND JAVA

DESCRIPTION AND STRUCTURE

THIS BACKEND IS BUILT USING SPRING BOOT, BELOW ARE THE KEY COMPONENTS AND FEATURES OF THE PROJECT:

DRIVERS

- AUTHCONTROLLER:
 - AUTHENTICATE METHOD
 - GETCURRENTUSER METHOD
- REGISTRATIONCONTROLLER:
 - REGISTERUSER METHOD

SERVICES

- JWTSERVICE
- USER SERVICE

MAIN DIRECTORY

- /BACKENDJAVA/DEMO

FILTERS

- JWTAUTHFILTER

KEY FILES

- POM.XML
- APPLICATION.PROPERTIES
- COM.EXAMPLE.DEMO

DEPENDENCIES

- SPRING BOOT
- JWT (TOKENS WEB JSON)
- LOMBOK

SECURITY SETTINGS

- CORSCONFIG
- SECURITYCONFIG

BACKEND JAVA

MAVEN CONFIGURATION

- MAVEN-WRAPPER.PROPERTIES
- MAVEN-COMPILER-PLUGIN

ADDITIONAL FEATURES

- ERROR HANDLING
- CONSOLE IMPRESSIONS

BACKEND PYTHON

STRUCTURE

THIS BACKEND IS BUILT USING THE FLASK FRAMEWORK AND IS DESIGNED TO HANDLE PRODUCT AND PURCHASING RELATED OPERATIONS.

MAIN FILES

- APP.PY
- REQUIREMENTS.TXT
- UPLOADS/

MAIN DIRECTORY

- /BACKENDPYTHON

ERROR HANDLING

- RESPONSES AND JSON FILES

SERVER CONFIGURATION

- FLASK
- CORS(CROSS-ORIGIN RESOURCE SHARING)

FILE MANAGEMENT

- UPLOAD DIRECTORY

DEPENDENCIES

- FLASK
- FLASK-CORS
- WERKZEUG

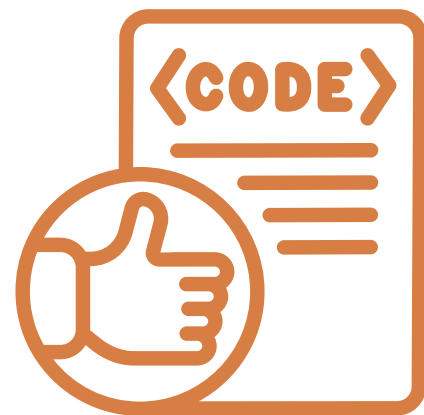
BACKEND PYTHON

ROUTES AND FUNCTIONALITIES

- MAIN ROUTE (/)
- PRODUCT MANAGEMENT (/PRODUCTS)
- GET PRODUCT BY ID
(/PRODUCTS/<INT:PRODUCT_ID>)
- PRODUCT REFERENCE MANAGEMENT
(/PRODUCTS/<INT:PRODUCT_ID>/REFERENCES)
- PURCHASE MANAGEMENT (/PURCHASE)
- PATH TO OBTAIN PURCHASES (/PURCHASES)

ADDITIONAL FEATURES

- MEMORY STORAGE
- VALIDATIONS



S

Java (Spring Boot) has separate classes;
Python (Flask) should split modules.

O

Java services extend functionality easily;
Python needs better structure.

L

Java interfaces allow flexible replacements;
Python should use inheritance.

I

Java controllers expose only required methods;
Python routes should be modular.

D

Java uses dependency injection;
Python benefits from blueprints/factory.

CONCLUSIONS



- **Well-Structured Backend:** Java and Python ensure functionality; better modularity improves maintenance.
- **SOLID Application:** Clear architecture enhances responsibility management.
- **User Stories and Diagrams:** Helped design relevant features and streamline development.
- **Mockups:** Improved UI early, reducing later changes.
- **Future Improvements:** Focus on modularity, documentation, and integration.